Itnavideo `audioClean` — Comprehensive Audit & Implementation Report

The specification describes a credible **AI-assisted voiceover cleanup** feature, but it is currently more of a marketing-level feature brief than an implementation-ready product specification. Its largest gaps are **non-destructive editing and undo/redo, transcript-to-audio edit integrity, A/B synchronization semantics, job lifecycle/resumability, credit failure handling, accessibility, and mobile interaction design**. The recommendations below are designed to be handed directly to Antigravity as implementation instructions.

Executive assessment

Area	Current maturity	Principal issue	Required action
Product concept	Strong	“Cleaner” conflates timing cleanup, retake selection, noise reduction, and mastering	Separate editing decisions from rendering/mastering
AI capability	Medium	“Retake detection” and “best performance” are under-specified	Define detection confidence, user approval, and edit provenance

Desktop UX	Medium	Transcript editor, waveform, and timeline relationship is undefined	Build a synchronized three-pane studio
Mobile UX	Missing	No mobile-specific layout or touch interactions	Create a mobile-first progressive workflow
Technical architecture	Weak	No job model, storage, idempotency, retry, or concurrency rules	Add durable job pipeline and state machine
Error handling	Basic	Only two explicit failure paths	Add exhaustive failure taxonomy and recovery flows
QA	Weak	Only three test statements	Add functional, AI-quality, accessibility, and performance test matrix
Accessibility	Missing	No keyboard, screen-reader, or transcript requirements	Meet WCAG 2.2 AA patterns

The existing spec correctly identifies the core value: preserve the creator's voice while removing dead air and failed takes. However, it does not define **what the user can safely change**, **what happens when AI is wrong**, or **how an edit maps back to exact source audio timestamps**. Those are the product's highest-risk requirements.

Critical product gaps

1. Define the editing model

The spec says users can “fix typos, remove sections, or paste an exact script,” but it does not explain whether transcript edits change the audio.

Problem: If a user deletes a transcript sentence, does the system remove the corresponding audio? If a user rewrites a word, does it regenerate speech? If it merely corrects display text, the user will reasonably expect audio deletion and be disappointed.

Required change: Introduce an explicit, non-destructive **Edit Plan** model.

Implement the following data model:

```ts
type AudioEditAction =
  | 'KEEP'
  | 'REMOVE'
  | 'SHORTEN_PAUSE'
  | 'SPLIT'
  | 'MERGE'
  | 'RETAKE_SELECT'
  | 'MANUAL_TRIM';

interface TranscriptSegment {
  id: string;
  text: string;
  startMs: number;
  endMs: number;
  confidence: number;
  speakerId?: string;
```

```
  isRetakeCandidate: boolean;
  retakeGroupId?: string;
  selectedTakeId?: string;
  userAction: AudioEditAction;
  sourceAudioRange: {
    startMs: number;
    endMs: number;
  };
}

interface AudioCleanEditPlan {
  version: number;
  sourceFileId: string;
  transcriptSegments: TranscriptSegment[];
  pauseEdits: PauseEdit[];
  retakeSelections: RetakeSelection[];
  masteringPresetId: string;
  outputFormat: 'MP3_320' | 'WAV' | 'M4A';

}
```

Why: This makes every user decision explicit, reversible, and renderable. It also prevents ambiguous behavior where transcript edits silently alter audio.

Implementation instructions for Antigravity:

- Store the original uploaded file immutably; never overwrite it.
- Generate a transcript with word- or phrase-level timestamps, not only sentence-level text.
- Make every transcript edit produce a structured `userAction`, not merely modified text.
- Distinguish **text correction** from **audio removal**:
  - Correcting “teh” to “the”: update transcript only.
  - Removing a sentence: mark its source audio range `REMOVE`.
  - Removing a section: mark all child segments `REMOVE`.
  - Pasting a script: do **not** assume it maps to source audio; open a mapping/review state.

- Render output only from the immutable source plus the saved `AudioCleanEditPlan`.
- Add a visible “Edits affect audio” indicator whenever a transcript deletion or retake selection changes the output.

2. Make retake detection reviewable

“Automatically detects repeated sentences” and “preserving only the best performance” are major claims without acceptance criteria.

Problem: Automatic selection of a “best” take is subjective and error-prone. A wrong deletion can remove intended content, damage trust, and create support escalations.

Required change: Make AI detection **assistive**, not autonomous.

Implementation instructions for Antigravity:

- Detect retake candidates using:
  - Semantic/text similarity between consecutive segments.
  - Acoustic similarity.
  - Repeated n-grams and near-duplicate sentence embeddings.
  - Hesitation markers such as “um,” “uh,” false starts, restarts, and repeated openings.
- Assign each candidate:
  - `retakeGroupId`
  - `confidence`: `high | medium | low`
  - `reason`: for example, “Repeated opening phrase detected”
  - `recommendedTake`
- Default behavior:
  - High confidence: pre-select the recommended take but allow one-click restore.
  - Medium confidence: show as a reviewable suggestion.
  - Low confidence: do not remove automatically.
- Never use the phrase “best performance” as an absolute claim in UI copy. Use “Recommended take” or “Suggested clean take.”
- Add a **Retake Review** panel:
  - Group repeated takes.
  - Play each candidate in isolation.

        - Show start/end time, duration, transcript, and confidence.
        - Allow “Keep,” “Remove,” “Play,” and “Restore all.”
- Provide a global **Restore all AI removals** action.

Why: This preserves user agency and prevents irreversible-feeling AI mistakes.

3. Add undo, redo, and version history

The spec has no undo/redo requirement. Audio editing is inherently exploratory, and users must be able to reverse decisions.

Problem: Without undo/redo, users may fear making edits, especially when transcript edits affect audio.

Required change: Implement an immutable edit-history stack.

Implementation instructions for Antigravity:

- Use a command-pattern state store for all edit actions.
- Persist an append-only edit history in client state and, after save, server-side draft state.
- Support:
    - Undo: `Cmd/Ctrl + Z`
    - Redo: `Cmd/Ctrl + Shift + Z`
    - Undo/redo buttons in the sticky action bar
    - History panel with labeled actions
- Each history entry should contain:
    - Action type
    - Affected segment IDs
    - Previous and next state
    - Timestamp
    - Short human-readable label
- Do not regenerate cloud audio for undo/redo. Undo/redo changes only the edit plan; regeneration is required only when the user requests a new preview/master.
- Add “Reset all edits” with confirmation and an explicit warning that it restores the original AI recommendations, not necessarily the original uploaded audio.

Why: Professional editors expect reversible, incremental editing. Adobe Audition's model separates editing from permanent application, which is the correct mental model here.

4. Clarify "mastering" claims

The spec promises "24-bit studio normalization, de-essing, room echo reduction," but does not define intensity, defaults, user control, or quality validation.

Problem: Aggressive de-essing, noise reduction, or normalization can create artifacts, pumping, dull consonants, or unnatural voice quality. This directly conflicts with the promise to preserve authentic vocal character.

Required change: Reframe mastering as a **conservative, user-controlled enhancement stage**.

Implementation instructions for Antigravity:

- Replace a single opaque "Broadcast Audio Mastering" toggle with a mastering preset system:
  - `Off / Original character`
  - `Gentle Cleanup`
  - `Podcast / Voiceover`
  - `Social Reel`
  - `Custom`
- Expose only understandable controls:
  - Background noise reduction: `Off | Low | Medium | High`
  - Room echo reduction: `Off | Low | Medium | High`
  - De-essing: `Off | Low | Medium | High`
  - Loudness normalization target: `-16 LUFS` for spoken content, `-14 LUFS` optional for social distribution
  - True peak ceiling: `-1.0 dBTP`
- Show a clear warning when high noise reduction or de-essing is selected: "Higher settings may affect natural voice texture."

- Preserve source sample rate where practical; do not claim “24-bit studio” unless the source and export pipeline genuinely support it.
- Add an A/B comparison specifically for mastering, separate from timing cleanup:
  - `Original`
  - `Timing cleaned`
  - `Timing cleaned + mastered`
- Add a “Reset mastering to defaults” control.

Why: Users need predictable control over audio character. The product should not imply that automatic processing is always desirable.

Workflow improvements

Current workflow issues

The proposed four-step flow is logical, but it hides important decisions and creates a risky “review then process” sequence.

Current step	Gap	Recommended change
Upload	No duration estimate, processing-time expectation, or upload recovery	Add file validation, progress, cancellation, and draft persistence
Configure	Pacing and format are bundled with processing	Let users configure before or after transcript review
Review transcript	No distinction between transcript correction and audio removal	Add explicit edit modes and visual audio impact

Preview/download	A/B preview may not reflect final mastering	Separate draft preview from final render

Recommended workflow

Implement a five-stage workflow:

1. **Upload & Analyze**
2. **Review Transcript & Retakes**
3. **Tune Pacing & Cleanup**
4. **Preview & Compare**
5. **Export & Download**

Implementation instructions for Antigravity:

- Allow non-linear navigation after analysis completes; do not force users through a rigid wizard.
- Persist draft state locally and server-side:
  - Uploaded source file reference
  - Transcript
  - Edit plan
  - Pacing profile
  - Mastering settings
  - Last preview render
- Show a persistent draft indicator: “Draft saved 2 minutes ago.”
- Add **Save draft**, **Discard draft**, and **Duplicate project** actions.
- Support returning to an unfinished project from the dashboard.
- Show estimated processing time before final render, based on source duration and selected operations.
- Prevent accidental navigation away during an active render with an unsaved-changes/processing warning.

Why: Creators often iterate across sessions. A rigid four-step flow that loses context after refresh would feel fragile.

AI capability requirements

Transcript quality

The specification assumes transcription will be accurate, but does not define language support, speaker handling, punctuation, timestamps, or correction workflow.

Implementation instructions for Antigravity:

- Detect or allow explicit selection of:
  - Language
  - Number of speakers: `Auto`, `1`, `2`, `Custom`
  - Domain vocabulary, if supported
- Produce:
  - Sentence-level transcript
  - Word-level timestamps where the provider supports them
  - Confidence score per segment
  - Alternative transcription when confidence is low
- Highlight low-confidence words or segments visually, but do not use color alone.
- Add “Mark as incorrect” and “Suggest correction” actions.
- Preserve original transcript text in an `originalText` field; store user edits separately in `editedText`.
- Do not silently overwrite the original transcript.
- Add a transcript export option: `TXT`, `SRT`, `VTT`, and `JSON`.

Why: Transcript corrections must be auditable, and low-confidence regions need user review rather than blind automation.

Silence detection

The current rule—trim pauses over 300 ms to a fixed gap—can produce unnatural speech, especially in interviews, emotional narration, comedy, and instructional content.

Implementation instructions for Antigravity:

- Do not use only fixed-duration thresholds.
- Classify pauses using:
  - Duration

        - Speech-to-speech context
        - Background noise level
        - Sentence boundary
        - Speaker change
        - Breath and room tone presence
- Preserve intentional pauses:
    - Dramatic pauses
    - Question-to-answer gaps
    - Speaker-change gaps
    - Pauses followed by laughter, music, or sound effects
- Add a “Protect pauses” control:
    - `Auto`
    - `Preserve dramatic pauses`
    - `Preserve speaker-change gaps`
    - `Custom`
- Add a previewable pause list with:
    - Start/end timestamp
    - Original duration
    - Proposed output duration
    - Classification: `Breath`, `Filler`, `Dead air`, `Speaker change`, `Dramatic pause`
    - Keep / shorten / restore action
- Add a “Restore all pauses” action.

Why: A fixed 100 ms gap may make a podcast sound rushed and robotic. The product should optimize for natural pacing, not merely maximum time saved.

Background-noise cleanup

The overview promises removal of “background hum,” but the feature list does not explain how this is detected or controlled.

Implementation instructions for Antigravity:

- Add an input-quality analysis stage:
    - Signal-to-noise ratio estimate
    - Clipping detection
    - Background noise estimate

        - Room reverb estimate
        - Plosive detection
        - Sibilance estimate
- Show a concise **Audio Health** panel:
    - `Good`, `Fair`, `Needs attention`
    - Detected issues
    - Recommended settings
- Never automatically apply destructive enhancement without user consent.
- Provide per-issue toggles:
    - Hum removal
    - Hiss reduction
    - Room echo reduction
    - De-essing
    - Plosive reduction
    - Loudness normalization
- Preserve an unprocessed original export option.

Why: Users need to understand what the AI will change before it changes it.

Desktop UI/UX improvements

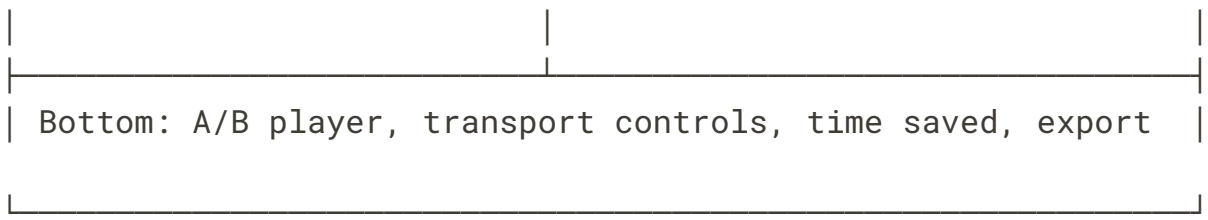
Layout architecture

The current “3-row layout” is too abstract for a professional editing studio. A fixed vertical flow will make transcript editing, waveform inspection, and A/B comparison inefficient.

Recommended desktop layout:

```
text
```

```
┌──────────────────────────────────────────────────────────────┐
| Top bar: project name, save state, undo/redo, help, export |
├──────────────────────────────┬───────────────────────────────┤
| Left: Transcript /           | Right: Waveform + Pause/Retake |
| Retake review                | Inspector                      |
```



Implementation instructions for Antigravity:

- Build a responsive three-region studio layout:
  - Left: transcript and retake review.
  - Right: waveform, pause inspector, and edit inspector.
  - Bottom: persistent player and action bar.
- Use collapsible panels, not hidden tabs, so users can see transcript and waveform together.
- Add resizable panels with sensible minimum widths.
- Keep the player visible during all editing actions.
- Add keyboard shortcuts:
  - `Space`: play/pause
  - `J/K/L`: rewind/pause/forward
  - `Arrow Left/Right`: seek
  - `Shift + Arrow`: select segment
  - `Delete`: remove selected segment
  - `Cmd/Ctrl + Z`: undo
  - `Cmd/Ctrl + Shift + Z`: redo
  - `B`: toggle before/after
- Add a command palette or keyboard-shortcut help modal.
- Show tooltips for every ambiguous control.
- Use a visible focus ring that meets contrast requirements.
- Avoid relying on hover-only affordances.

Why: Desktop users expect an efficient, keyboard-driven editing surface. The current spec does not define any of these interactions.

Waveform and timeline

The spec mentions a “Timeline Pause Inspector,” but does not define waveform behavior, zoom, selection, or scrubbing.

Implementation instructions for Antigravity:

- Render two synchronized waveform lanes:
  - `Original`
  - `Cleaned preview`
- Use distinct but accessible visual treatments:
  - Original: neutral slate/gray
  - Cleaned: emerald green
  - Removed region: diagonal hatch or reduced opacity, plus a text label
- Do not communicate state using color alone.
- Add:
  - Playhead
  - Click-to-seek
  - Drag-to-scrub
  - Zoom in/out
  - Fit-to-project
  - Keyboard panning
  - Segment selection
  - Play selection loop
- Show removed regions with:
  - Start/end time
  - Duration removed
  - Reason: `Pause`, `Retake`, `Manual removal`
  - Restore action
- Add a minimap overview for long recordings.
- Maintain scroll position when toggling A/B modes.

Why: Users must understand exactly which milliseconds are being removed. A transcript alone is insufficient for audio-editing confidence.

A/B comparison

The current “Live A/B Comparison Switcher” is promising but underspecified.

Problem: “Synchronized timeline scrubbing” is ambiguous. If the cleaned audio has removed time, its timeline cannot be identical to the original.

Required change: Define two synchronization modes.

Implementation instructions for Antigravity:

- Implement **Content-Sync mode** as default:
  - When the user switches from Before to After, playback continues at the corresponding spoken content position.
  - Example: if the original is at 00:01:20 and 8 seconds of silence were removed before that point, the cleaned version should start near 00:01:12.
- Implement **Absolute-Time mode** as an advanced option:
  - Both players use the same wall-clock timestamp.
  - Useful for auditing exact edits.
- Show a clear mode label:
  - `Content sync`
  - `Absolute time`
- When switching modes, show a brief non-blocking explanation.
- Preserve:
  - Play/pause state where technically feasible
  - Volume
  - Playback rate
  - Loop state
- Add a visual diff ribbon showing removed regions.
- Add a “Play selected edit” action that plays only the boundary region before and after a cut.

Why: Users need to judge whether a cut sounds natural. A naive time-sync implementation will mislead them because removed audio shifts the cleaned timeline.

Mobile UI/UX improvements

The specification has no mobile requirements, despite the product targeting creators who may upload and review from phones.

Mobile-first strategy

Do not simply compress the desktop studio into a narrow viewport. Mobile should prioritize **review, correction, and export**, while complex waveform editing remains functional but simplified.

Recommended mobile layout:

```
text
```

```
┌──────────────────────────────────┐
| Project header + Save state  |
├──────────────────────────────────┤
| Primary content card         |
| Transcript / Retakes /       |
| Pause list / Waveform        |
├──────────────────────────────────┤
| Bottom sheet: A/B + Export   |

└──────────────────────────────────┘
```

Implementation instructions for Antigravity:

- Use a tabbed or segmented mobile workspace:
  - `Transcript`
  - `Retakes`
  - `Pauses`
  - `Sound`
  - `Export`
- Keep a persistent bottom action bar with:
  - Play/Pause
  - Before/After toggle
  - Current time
  - Primary CTA
- Use a bottom sheet for:
  - Pacing profile
  - Output format
  - Mastering settings
  - Segment actions
  - Export confirmation
- Make all primary tap targets at least 44 × 44 CSS pixels.
- Avoid hover-dependent controls.
- Use native-feeling swipe gestures only where they are discoverable:
  - Swipe left on a transcript segment: reveal `Remove`, `Keep`, `Play`
  - Swipe right: reveal `Restore` where applicable

- Provide visible buttons in addition to swipe actions.
- Use sticky headers sparingly; avoid consuming excessive vertical space.
- Ensure the virtual keyboard does not obscure the transcript editor.
- Auto-scroll the active transcript segment during playback.
- Allow users to tap a transcript sentence to seek to its audio position.
- Show a compact waveform with pinch-to-zoom and drag-to-scrub.
- Provide a simplified “Review suggested edits” mode for mobile:
    - Accept all high-confidence recommendations
    - Review each suggestion individually
    - Restore all AI edits

Why: Mobile users should be able to complete the core job—review, adjust, preview, and export—without needing a desktop.

Mobile upload experience

Implementation instructions for Antigravity:

- Support camera-roll/file-picker upload and direct in-browser recording.
- Show:
    - File name
    - Size
    - Duration
    - Upload progress
    - Cancel button
    - Retry state
- Validate before upload:
    - Supported MIME types
    - Maximum file size
    - Maximum duration
    - Minimum viable audio level
- Warn early when a file exceeds 5 minutes:
    - “This recording is 7 minutes 12 seconds. The current limit is 5 minutes.”
- Do not begin credit-consuming processing until the user explicitly confirms the final render.

Why: Mobile network conditions and file-picker behavior make robust upload handling essential.

Technical architecture gaps

Missing job lifecycle

The spec says processing is cloud-based but provides no job model, queue behavior, status states, retry policy, or artifact lifecycle.

Implementation instructions for Antigravity:

Implement a durable processing pipeline with explicit states:

```ts
type AudioCleanJobStatus =
  | 'QUEUED'
  | 'UPLOADING'
  | 'VALIDATING'
  | 'EXTRACTING_AUDIO'
  | 'TRANSCRIBING'
  | 'ANALYZING_PAUSES'
  | 'DETECTING_RETAKES'
  | 'AWAITING_USER_REVIEW'
  | 'RENDERING_PREVIEW'
  | 'RENDERING_FINAL'
  | 'UPLOADING_OUTPUT'
  | 'COMPLETED'
  | 'FAILED'
  | 'CANCELLED'

  | 'EXPIRED';
```

Required backend behaviors:

- Use an idempotency key for every render request.
- Store job state in a durable database, not only in memory.

- Use a queue with visibility timeout, retry policy, and dead-letter handling.
- Make each worker step idempotent.
- Store intermediate artifacts with signed, expiring URLs.
- Separate:
    - Source upload
    - Audio extraction
    - Transcription
    - Analysis
    - Preview render
    - Final render
    - Output upload
- Add progress percentages per stage.
- Add a cancellation token that propagates through the pipeline.
- Add automatic retry for transient provider failures, with exponential backoff and a maximum retry count.
- Surface actionable errors to users; never expose raw provider stack traces.
- Retain output artifacts according to a defined retention policy, for example 7 days, with explicit user-facing expiry messaging.

Why: Cloud FFmpeg-style workloads require explicit handling of queues, retries, temporary storage, and signed delivery; these are absent from the current spec.

Processing architecture

Implementation instructions for Antigravity:

- Run transcription, analysis, and rendering as separate, observable pipeline stages.
- Cache transcription results by a stable hash of:
    - Source file content hash
    - Audio extraction settings
    - Transcription model/version
    - Language and speaker settings
- Do not re-transcribe when only pacing or mastering settings change.
- Use server-side FFmpeg or an equivalent managed media-processing service.
- Generate a **preview render** using fast, lower-cost processing where appropriate.
- Generate the final master only after explicit user confirmation.
- Store both:

    - `previewUrl`
    - `finalOutputUrl`
- Use signed, short-lived download URLs.
- Add virus/malware and file-integrity validation before processing.
- Enforce server-side duration and file-size limits; do not rely on client-side validation.
- Stream or chunk large uploads; do not buffer entire videos in memory.
- Add rate limiting per user, IP, and workspace.
- Add observability:
    - Job duration by stage
    - Provider error rate
    - Retry rate
    - Transcription confidence distribution
    - Render failure rate
    - Credit refund rate
    - Output download conversion rate

Why: The current specification names technologies but not the production guarantees needed for a reliable AI media product.

Data and privacy

Implementation instructions for Antigravity:

- State clearly whether uploaded audio/video is used for model training.
- Provide a data-retention setting or policy link before upload.
- Delete source and derivative artifacts according to the stated retention policy.
- Encrypt files at rest and in transit.
- Restrict access using workspace/user authorization.
- Log processing events without storing raw audio in logs.
- Provide a “Delete project and files” action with confirmation.
- Do not retain transcript content longer than necessary.

Why: Voice recordings are sensitive biometric-adjacent content. Privacy expectations are especially high for voice data.

Business-rule gaps

Credit logic is incomplete

The spec says “strictly 1 credit per cleaned audio file,” but does not define when the credit is consumed, refunded, or reused.

Implementation instructions for Antigravity:

Define credit events precisely:

Event	Credit behavior
Upload and analysis	No charge
Transcript correction	No charge
Pacing/profile change	No charge
Preview render	No charge, subject to fair-use limits
Final download/render	1 credit
Final render failure caused by Itnavideo	Automatic refund or no charge
User cancels before final render	No charge

User cancels after final render starts	Product decision required; document it
Re-render after changing pacing	Define whether this consumes another credit
Re-render after changing mastering only	Define whether this consumes another credit
Downloading the same completed output again	No additional credit
Exporting multiple formats from one completed render	Define policy

Required implementation instructions:

- Charge credits only after a final render successfully completes and is available for download.
- Use an atomic credit reservation-and-settlement pattern:
    1. Reserve 1 credit.
    2. Start render.
    3. On success, settle the reservation.
    4. On failure, release/refund the reservation.
- Never double-charge on retry.
- Store a `creditTransactionId` on every render job.
- Show the credit cost before the user clicks **Download Cleaned Audio**.
- Show a confirmation modal if the user has insufficient credits.
- Add a clear pricing/help link beside the credit indicator.

Why: “1 credit per cleaned audio file” is ambiguous when users re-render or export multiple formats.

Duration limit is too rigid

A hard 5-minute cap may be acceptable for an initial release, but it excludes podcast segments, lectures, interviews, and longer voiceover sessions.

Implementation instructions for Antigravity:

- Keep 5 minutes as the initial processing limit if required by cost or infrastructure.
- Clearly distinguish:
  - `Maximum source duration`
  - `Maximum output duration`
  - `Maximum file size`
  - `Maximum concurrent jobs`
- Show remaining allowed duration before upload.
- Add an upsell or future roadmap message for longer recordings, but do not make misleading claims.
- For video inputs, extract audio and clearly tell users that video is not returned as output.
- Add a “trim to first 5 minutes” option when a longer file is uploaded, with explicit user consent.

Why: Users should never discover limits after waiting for an upload or processing job.

Error handling and edge cases

The current error section covers only no-speech detection and memory cleanup. It needs a complete failure taxonomy.

Required error states

Scenario	User-facing message	System behavior	Credit impact
Unsupported file type	“Use MP3, WAV, M4A, AAC, MP4, or MOV.”	Block upload	None
Corrupt/unreadable media	“We couldn’t read this file. Try exporting it again or upload a different file.”	Fail validation	None
File too large	Show actual vs. allowed size	Block upload	None
Recording longer than 5 minutes	Show duration and trimming option	Block or offer trim	None
No speech detected	“We couldn’t detect clear speech. Upload a recording with audible speech.”	No render	None
Mostly music/noise	“This sounds like music or background noise rather than speech.”	Offer manual review or reject	None

Multiple speakers	“We detected multiple speakers. Review speaker labels before cleanup.”	Enable speaker review	None
Transcription provider failure	“Transcription is temporarily unavailable. Your draft is saved—try again.”	Retryable	None
Transcription quality too low	“We couldn’t create a reliable transcript. You can still use manual silence trimming.”	Offer limited mode	No final charge
Render provider failure	“We couldn’t finish processing. Nothing was charged.”	Retry/refund	Refund/no charge
Output upload failure	“Your cleaned audio was created but couldn’t be saved. Please retry.”	Retry	Refund/no charge
Network interruption during upload	“Upload interrupted. Resume or start again.”	Resumable if feasible	None
Stale/expired output URL	“This download link expired. Generate a new download.”	Re-sign URL	No new credit

Concurrent render request	“A render is already in progress for this project.”	Block duplicate	No duplicate charge
Quota exceeded	“You’ve reached your credit limit.”	Show upgrade/top-up path	None
Empty transcript edit	“Add or restore content before exporting.”	Block export	None
All audio removed	“No audio remains after your edits.”	Block export	None
Extreme silence trimming	“This pacing may sound unnatural. Try Natural Flow.”	Advisory	None

Implementation instructions for Antigravity:

- Map every internal error code to a user-safe message.
- Include a support/reference ID for server-side failures.
- Add retry buttons only where retry is safe.
- Preserve user edits across recoverable failures.
- Add telemetry for every error code, provider, file type, duration bucket, and retry outcome.
- Add a status page or in-app incident banner for provider outages.

Accessibility requirements

The specification does not address accessibility. Because this product centers on audio, accessibility is not optional.

Implementation instructions for Antigravity:

- Provide an accessible HTML transcript alongside the player.
- Ensure all player controls are keyboard accessible:
  - Play/pause
  - Seek
  - Volume/mute
  - Playback speed
  - Before/After toggle
  - Segment selection
- Provide visible focus indicators and logical tab order.
- Use `aria-live` announcements for:
  - Upload progress
  - Analysis completion
  - Render completion
  - Error messages
  - Undo/redo actions
- Label all icon-only buttons with accessible names.
- Do not communicate removed audio solely through color.
- Ensure text contrast meets WCAG AA.
- Support `prefers-reduced-motion`.
- Avoid autoplay; require explicit user action to play audio.
- Provide downloadable transcript formats.
- Ensure the transcript editor is usable with screen readers, including:
  - Segment role and position
  - Edit action announcement
  - Removed/restored state
  - Confidence warning

WCAG requires text alternatives for time-based media, and audio-only content should have an accessible transcript; player controls should also be keyboard operable and clearly labeled.

Testing requirements

The current QA section is insufficient for production. It checks type safety and two subjective behaviors, but not the full product surface.

Required test matrix

Area	Required tests
Upload	Valid/invalid MIME types, oversized files, zero-byte files, corrupt files, long files, video files, interrupted uploads, duplicate uploads
Transcription	English and non-English speech, accented speech, multiple speakers, overlapping speech, low-quality audio, music-heavy audio, silence-only audio
Retake detection	True positives, false positives, near-duplicate sentences, intentional repetition, low-confidence candidates, restore-all behavior
Pause trimming	Short pauses, long pauses, breaths, speaker changes, dramatic pauses, background noise, cross-fade artifacts
Transcript editing	Typo correction, sentence deletion, section deletion, script paste, undo/redo, invalid mapping, all-audio-removed
A/B player	Content sync, absolute time, seek, loop, playback-rate change, browser tab backgrounding, mobile interruption

Mastering	Each preset, each intensity, clipped input, quiet input, noisy input, sibilant speech, reverb-heavy speech
Export	MP3 320 kbps, WAV, M4A, filename handling, download retry, expired URL, repeated download
Credits	Successful render, failed render, retry, concurrent render, insufficient credits, refund, duplicate download
Performance	30-second, 1-minute, 3-minute, and 5-minute files; slow network; large video input
Accessibility	Keyboard-only operation, screen-reader announcements, focus order, reduced motion, color independence
Responsive	320 px, 375 px, 768 px, 1024 px, 1280 px, 1920 px; landscape mobile; tablet split view
Resilience	Provider timeout, provider 429/5xx, queue backlog, job restart, expired artifact, cancel during each stage

Acceptance criteria

Add these explicit acceptance criteria to the specification:

- A user can undo every edit action and return to the prior edit-plan state.
- A user can restore any AI-removed segment or pause.

- A user can distinguish transcript-only corrections from audio-removing edits.
- A/B switching preserves the corresponding spoken-content position in `Content sync` mode.
- A failed final render never consumes a credit.
- A user can re-download a successfully completed output without paying again.
- All primary actions are operable by keyboard.
- The mobile experience supports upload, review, basic editing, preview, and export.
- A 5-minute source completes analysis and provides actionable progress within a defined SLA.
- No output is generated without an explicit final-render action.

Priority implementation plan for Antigravity

P0 — Must fix before release

1. Implement the immutable source + editable `AudioCleanEditPlan` architecture.
2. Separate transcript correction from audio deletion.
3. Add undo, redo, restore-all, and edit history.
4. Define exact A/B synchronization behavior, especially `Content sync` mode.
5. Implement the full job state machine, idempotency, retries, cancellation, and signed output URLs.
6. Charge credits only after successful final render; implement atomic reservation/refund logic.
7. Add complete upload validation, progress, cancellation, and recovery.
8. Add accessible transcript, keyboard controls, focus states, and `aria-live` messaging.
9. Add mobile layouts, touch targets, bottom sheets, and mobile upload handling.
10. Add the complete error taxonomy and user-safe error messages.

P1 — High-value improvements

1. Add retake review with confidence, reason, and per-take playback.
2. Add pause classification and preservation controls.
3. Add audio-health analysis and user-controlled enhancement settings.
4. Add waveform zoom, selection, scrubbing, and removed-region visualization.
5. Add draft persistence, project resume, and duplicate-project support.
6. Add transcript export in `TXT`, `SRT`, `VTT`, and `JSON`.
7. Add mastering presets and an original-character preservation option.

8. Add comprehensive QA automation and AI-quality evaluation datasets.

P2 — Differentiators

1. Add speaker diarization and per-speaker editing.
2. Add filler-word removal with reviewable suggestions.
3. Add breath preservation and breath softening.
4. Add music/sound-effect protection.
5. Add batch processing for multi-file voiceover projects.
6. Add project templates for podcast, course, reel, and audiobook workflows.
7. Add collaborative review and comments.
8. Add localization and multilingual transcription quality controls.

Final implementation instruction for Antigravity

Implement `audioClean` as a **non-destructive, plan-based audio editing studio**, not as a one-shot “clean audio” black box.

The core architecture must be:

```
text

Immutable source media
        ↓
Cloud transcription + acoustic analysis
        ↓
Structured AudioCleanEditPlan
        ↓
User review, correction, undo/redo
        ↓
Fast preview render
        ↓
Explicit final render + credit settlement
        ↓

Signed, expiring output artifact
```

Every AI recommendation must be visible, explainable, reversible, and user-approved before it can affect the exported audio. Every cloud operation must be idempotent, observable, recoverable, and credit-safe. Every interface must work on desktop and mobile, with keyboard and screen-reader support as first-class requirements.

The most important product principle is: **the user remains the final editor; AI proposes, the user decides, and the system preserves the original.** This aligns with the product's stated promise of preserving the speaker's authentic voice while still giving creators meaningful control over pacing, retakes, cleanup, and mastering.